

UNIVERSITÀ DEGLI STUDI DI ROMA “TOR VERGATA”

Macroarea di Ingegneria

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

INGEGNERIA DEL SOFTWARE E PROGETTAZIONE WEB (ISPW)

CIBOAMICO

*PIATTAFORMA PER LA GESTIONE DEL CIBO IN CASA, LA RIDUZIONE
DEGLI SPRECHI E L'OTTIMIZZAZIONE DELLA SPESA*

DOCENTI:

PROF. DAVIDE FALESSI

PROF. GUGLIELMO DE ANGELIS

AUTORE:

MICHELE DAMIANO

MATRICOLA: 0324516

MICHELE.DAMIANO@STUDENTS.UNIROMA2.EU

ANNO ACCADEMICO 2025/2026

DATA DI CONSEGNA: 11 AGOSTO 2026

Indice

1	Specifiche dei Requisiti Software	3
1.1	Introduzione	3
1.1.1	Scopo del documento	3
1.1.2	Panoramica del sistema	3
1.1.3	Requisiti hardware e software	4
1.1.4	Sistemi correlati	4
1.2	User Stories	5
1.3	Requisiti Funzionali	5
1.3.1	Glossario dei Termini di Dominio	6
1.4	Use Cases	6
1.4.1	Diagramma dei casi d'uso	6
1.4.2	Internal Steps	7
2	Storyboards	9
2.1	Autenticazione e Area Personale	9
2.1.1	Autenticati	9
2.1.2	Home (Cliente)	9
2.2	Acquisti e Marketplace (UC-04 Ordina un Prodotto)	10
2.2.1	Marketplace (Catalogo Prodotti)	10
2.2.2	Ordina un Prodotto	11
2.2.3	Pagamento (passo 6)	12
3	Design	14
3.1	VOPC (Analysis)	14
3.2	Class Diagram (Design-level)	14
3.2.1	Controller	15
3.2.2	Validazione input e gestione errori	16
3.2.3	Design Patterns (GoF)	16
3.3	Modellazione Comportamentale	20
3.3.1	Activity Diagram (UC-04)	20
3.3.2	Sequence Diagram (UC-04)	21

3.3.3	State Diagram (UC-04, navigazione UI)	23
3.3.4	State Diagram (ciclo di vita dell'ordine, BR-04)	23
4	Testing	24
5	Exceptions	26
6	Persistenza	28
6.1	Layer di persistenza DB (MySQL/JDBC)	28
6.2	Layer di persistenza FS (File System)	28
6.3	Layer di persistenza DEMO (In-Memory)	29
7	Video e Link	30
7.1	Repository GitHub	30
7.2	SonarCloud	30
7.3	Video dimostrativo	30

Capitolo 1

Specifiche dei Requisiti Software

1.1 Introduzione

1.1.1 Scopo del documento

Questo documento definisce i requisiti software di **CiboAmico**, il progetto finale del corso ISPW (Università di Roma “Tor Vergata”, A.A. 2025/2026, Prof. Davide Falessi, Prof. Guglielmo De Angelis).

Lo scopo è fissare cosa il sistema deve fare in modo chiaro e verificabile, lasciando campo alla progettazione e ai test. Il documento serve sia a chi sviluppa, per tenere fede all’architettura Boundary-Control-Entity (BCE), sia a chi valuta il prodotto, per confrontare i flussi funzionali con le richieste iniziali.

1.1.2 Panoramica del sistema

CiboAmico è un’applicazione desktop JavaFX che aiuta a gestire il cibo in casa, evitare sprechi e ottimizzare la spesa, collegando l’utente ai commercianti di prossimità. Il sistema segue il pattern Boundary-Control-Entity (BCE) e coinvolge quattro attori: Cliente, Venditore, Nutrizionista e Amministratore. Due sistemi esterni sono raggiunti tramite Adapter: OpenFoodFacts (lettura dei dati nutrizionali dal codice a barre) e JakartaMail (invio di notifiche). La persistenza è intercambiabile a runtime tra una modalità *Demo* (in-memory) e una *Full* (MySQL via JDBC).

Il sistema ha tre aree funzionali:

- **Dispensa:** il Cliente traccia i prodotti in casa, vede le scadenze e viene avvisato prima che il cibo si deteriori;
- **Ricette:** il sistema propone ricette compatibili con gli ingredienti disponibili, sostituendo quelli mancanti;

- **Marketplace:** i Venditori pubblicano i prodotti, i Clienti fanno ordini diretti (un compratore, un venditore) e l'Amministratore approva venditori e ricette.

1.1.3 Requisiti hardware e software

L'applicazione è cross-platform grazie alla JVM.

Requisiti software:

- Java 17+
- JavaFX 17+ (per l'interfaccia desktop)
- Maven 3.9+ (build tool)
- MySQL Server 8.0+ (modalità *Full*, via JDBC)
- Draw.io (per la creazione dei diagrammi UML)
- IDE: IntelliJ IDEA (con EcEmma per la coverage) o Eclipse

Requisiti hardware:

- CPU: Dual-Core 2.0 GHz a 64-bit
- RAM: 2 GB (4 GB consigliati in modalità *Full* con server MySQL locale)
- Spazio su disco: 1 GB
- Schermo: 1024×768
- Sistema operativo: Windows, macOS, Linux (compatibile con Java 17+)

1.1.4 Sistemi correlati

Di seguito due sistemi esistenti che coprono solo in parte quello che fa CiboAmico.

System 1: Too Good To Go

Pro:

- Rete capillare di partner con offerte geolocalizzate in tempo reale.
- Riduce attivamente lo spreco commerciale offrendo cibo a prezzi ridotti.

Contro:

- Non traccia la dispensa di casa: dopo l'acquisto non aiuta a gestire le scadenze.
- L'acquisto è una *Magic Box* a sorpresa: non si scelgono i singoli prodotti né si pianificano ricette o diete.

System 2: SuperCook

Pro:

- Matching potente: trova subito molte ricette a partire dagli ingredienti inseriti.
- Ampio database con filtri per tipo di piatto e intolleranze.

Contro:

- Non ha un canale di acquisto integrato: per gli ingredienti mancanti bisogna provvedere da soli.
- Non sostituisce automaticamente gli ingredienti, escludendo ricette realizzabili con piccole variazioni.

CiboAmico punta a combinare tracciamento della dispensa, generazione di ricette con sostituzioni e acquisto diretto dai produttori locali in un'unica piattaforma, offrendo funzionalità dedicate a clienti, venditori, nutrizionisti e amministratori, mantenendo un'architettura modulare ed estensibile.

1.2 User Stories

1. **US-1:** Come Cliente, voglio ricevere notifiche preventive sulle date di scadenza dei prodotti presenti nella mia dispensa, in modo che possa consumare gli alimenti in tempo ed evitare lo spreco di cibo.
2. **US-2:** Come Cliente, voglio visualizzare le ricette compatibili con gli ingredienti disponibili in dispensa, in modo che possa cucinare senza dover effettuare nuovi acquisti.
3. **US-3:** Come Cliente, voglio ordinare un prodotto direttamente da un venditore locale, in modo che possa ricevere cibo fresco proveniente dal mio territorio.

1.3 Requisiti Funzionali

1. **RF-1:** Il sistema deve confrontare quotidianamente le date di scadenza degli alimenti nella dispensa con la data corrente, inviando un avviso al cliente per ogni prodotto la cui scadenza sia entro tre giorni.
2. **RF-2:** Il sistema deve suggerire al cliente un elenco di ricette in cui gli ingredienti richiesti siano un sottoinsieme degli ingredienti non scaduti presenti nella dispensa.
3. **RF-3:** Il sistema deve registrare una nuova transazione di acquisto associando il cliente al venditore e decrementando la scorta del prodotto in misura pari alla quantità ordinata.

1.3.1 Glossario dei Termini di Dominio

Dispensa : rappresentazione digitale degli alimenti presenti nell’inventario domestico dell’utente, con quantità e date di scadenza.

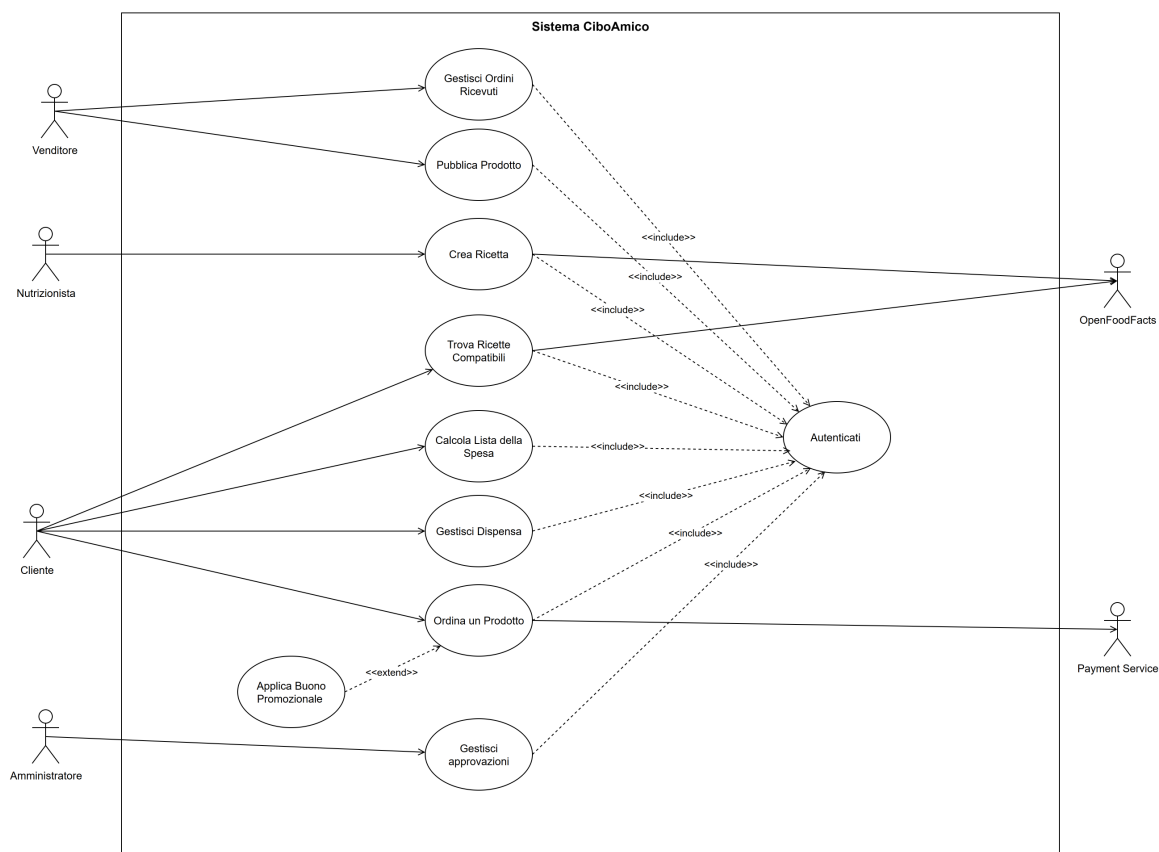
Ricetta compatibile : ricetta i cui ingredienti richiesti sono presenti, oppure sostituibili in modo equivalente, nella dispensa dell’utente.

Ordine diretto : transazione che associa un singolo cliente acquirente a un unico venditore, senza passare da un carrello condiviso o intermediari.

Disponibilità del prodotto : quantità di un prodotto attualmente vendibile, che non può mai essere negativa e viene aggiornata dopo ogni acquisto.

1.4 Use Cases

1.4.1 Diagramma dei casi d’uso



Il caso d’uso **Ordina un Prodotto** è stato progettato con un grado di astrazione che consente di modellare funzionalità distinte. In questa implementazione, il caso d’uso secondario **Applica Buono Promozionale** è associato al caso d’uso primario tramite una relazione di *extend*: l’attore Cliente può utilizzare questa funzionalità estesa solo se si

autentica correttamente nel sistema (relazione di *include* verso **Autenticati**) e proseguendo poi il flusso del caso d'uso **Ordina un Prodotto**. Analogamente, **Autenticati** è incluso da più casi d'uso, a sottolinearne il riuso nell'ambito delle funzionalità principali.

1.4.2 Internal Steps

Ordina un Prodotto

Scenario principale:

1. Il Sistema individua i prodotti disponibili sul marketplace con prezzi, scorte residue e venditori.
2. Il Cliente richiede di aggiungere un prodotto all'ordine, indicandone la quantità desiderata.
3. Il Sistema convalida che la quantità richiesta non superi la scorta residua del prodotto.
4. Il Sistema calcola l'importo totale della transazione.
5. Il Cliente richiede di confermare l'acquisto.
6. Il Sistema richiede al Payment Service l'autorizzazione all'addebito per l'importo calcolato.
7. Il Sistema registra l'ordine in attesa di approvazione e decrementa la scorta residua dei prodotti ordinati.
8. Il Sistema invia una notifica al Venditore per segnalare l'ordine ricevuto.

Estensioni:

- **3a. Superamento della scorta residua:**

1. Il Sistema rileva che la quantità inserita dal Cliente è superiore alla disponibilità corrente del prodotto.
2. Il Sistema segnala l'anomalia al Cliente indicando la massima quantità acquistabile.
3. Il flusso di controllo ritorna al passo 2.

- **4a. Richiesta di applicazione di un buono promozionale (tramite l'estensione del caso d'uso Applica Buono Promozionale):**

1. Il Cliente richiede di applicare un codice di buono promozionale all'ordine corrente.

2. Il Sistema convalida la validità temporale del buono promozionale e la sua associazione con il Venditore del prodotto selezionato.
 3. Il Sistema ricalcola l'importo totale della transazione applicando lo sconto previsto dal buono promozionale.
 4. Il flusso di controllo prosegue dal passo 5 dello scenario principale.
- **6a. Autorizzazione di pagamento negata:**
 1. Il Sistema riceve l'esito negativo dall'autorizzazione del Payment Service.
 2. Il Sistema segnala al Cliente il fallimento del pagamento.
 3. Il flusso di controllo ritorna al passo 5.

Capitolo 2

Storyboards

2.1 Autenticazione e Area Personale

2.1.1 Autenticati

La schermata di accesso permette all'utente di autenticarsi inserendo le proprie credenziali (email e password).

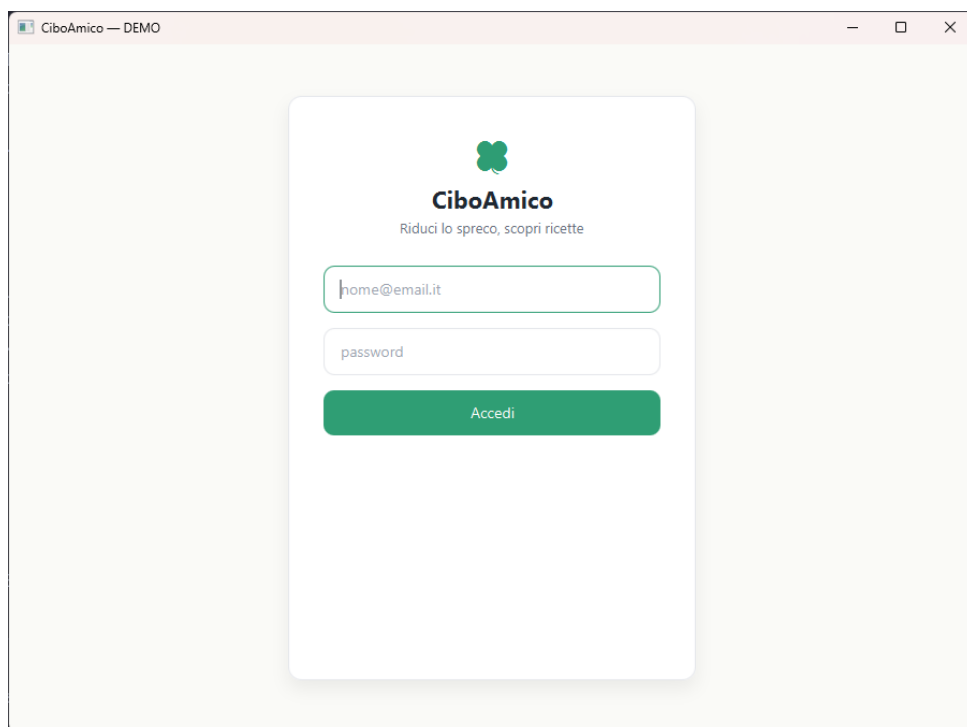


Figura 2.1: Schermata di autenticazione al sistema.

2.1.2 Home (Cliente)

Completata l'autenticazione, il Cliente raggiunge la dashboard principale (Home). Da qui può raggiungere le funzionalità dell'applicazione: Ricette, Inventario, Lista Spesa e

Marketplace (UC-04 Ordina un Prodotto).

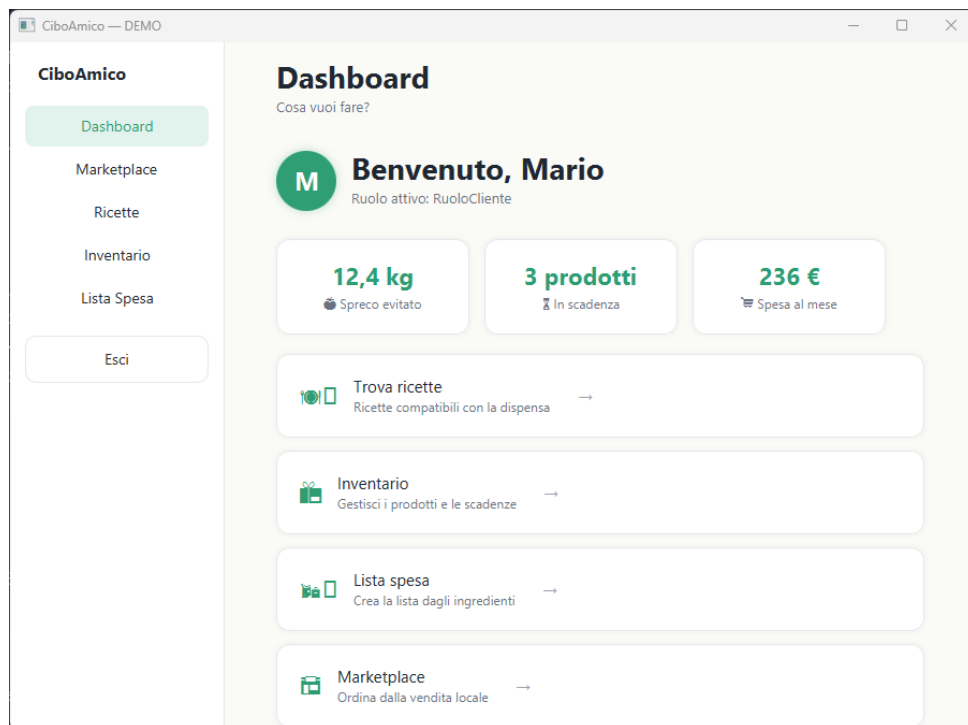


Figura 2.2: Dashboard principale: accesso alle funzionalità dell'applicazione.

2.2 Acquisti e Marketplace (UC-04 Ordina un Prodotto)

2.2.1 Marketplace (Catalogo Prodotti)

L'area dedicata all'acquisto diretto mostra il catalogo dei prodotti locali dei venditori. Il pannello laterale consente di selezionare un prodotto da ordinare e, opzionalmente, un buono promozionale.

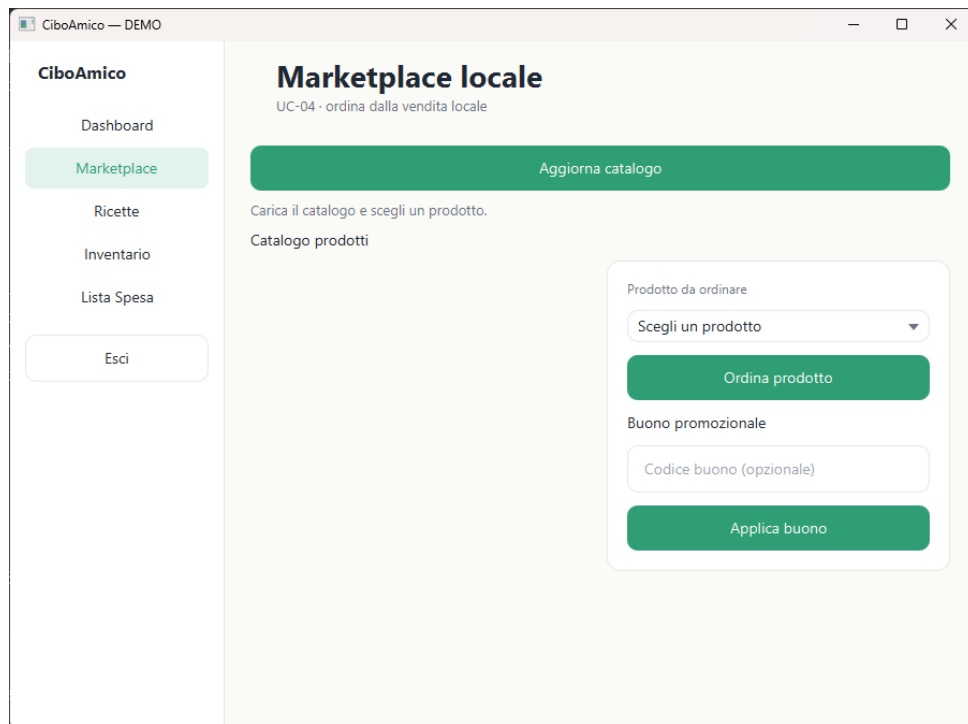


Figura 2.3: Marketplace: catalogo dei prodotti locali e pannello di acquisto.

2.2.2 Ordina un Prodotto

Selezionato un prodotto dal catalogo e caricata la lista (pulsante *Aggiorna catalogo*), il Cliente sceglie il prodotto nel pannello *Prodotto da ordinare*. Se inserisce un codice buono e preme *Applica buono*, il sistema valida il buono e ricalcola il totale applicando lo sconto (estensione *Ordina un Prodotto*); con *Ordina prodotto* procede.

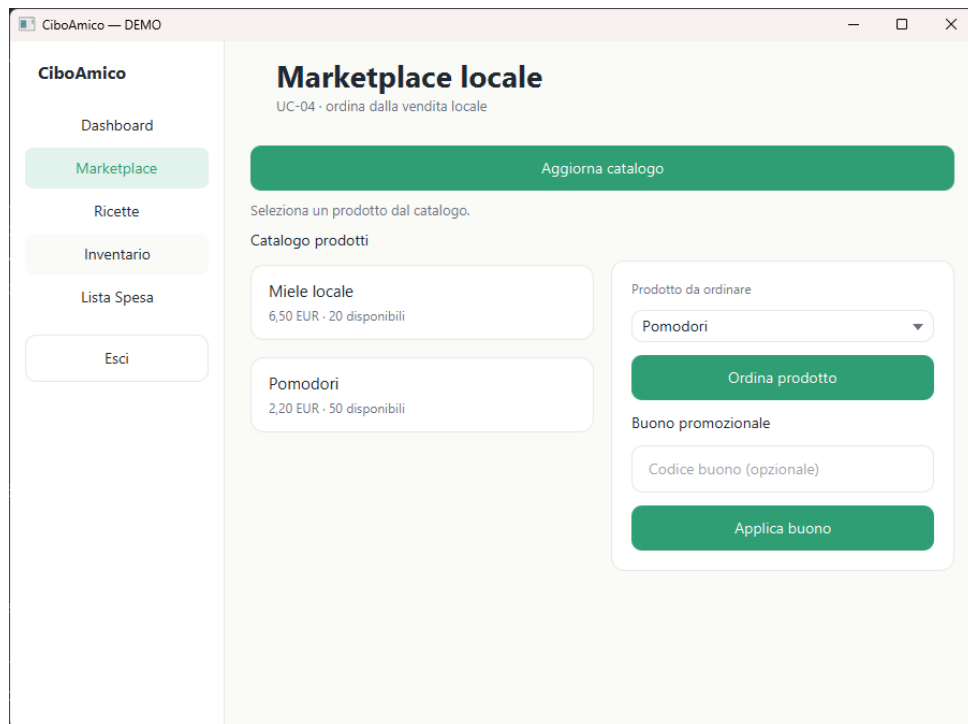


Figura 2.4: Schermata di acquisto con prodotto selezionato e campo buono promozionale.

2.2.3 Pagamento (passo 6)

Confermato l'ordine, l'interfaccia passa alla schermata di pagamento: l'utente inserisce i dati della carta (numero, intestatario, scadenza e CVV) e autorizza l'addebito; l'esito è mostrato inline nella stessa vista, come richiesto dall'estensione **6a** (pagamento rifiutato) del caso d'uso.

The screenshot shows a web application window titled "CiboAmico — DEMO". On the left is a sidebar menu with the following items: "Dashboard", "Marketplace" (highlighted in green), "Ricette", "Inventario", "Lista Spesa", and "Esci". The main content area is titled "Pagamento" and includes the subtitle "UC-04 · autorizzazione all'addebito". Below this, a summary box states "Riepilogo: Pomodori — 2,20 EUR". The payment form contains the following fields: "Numero carta" (with a placeholder "Numero carta"), "Intestatario" (with a placeholder "Intestatario"), "Scadenza" (with a placeholder "Scadenza (MM/AA)"), and "CVV" (with a placeholder "CVV"). At the bottom of the form are two large green buttons labeled "Paga" and "Annulla".

Figura 2.5: Schermata di pagamento (dati carta e autorizzazione all'addebito).

Capitolo 3

Design

Il capitolo illustra la progettazione del caso d'uso **Ordina un Prodotto** (UC-04): vista di analisi (VOPC), class diagram di progettazione con i pattern GoF applicati e modellazione comportamentale (activity, sequence, state).

3.1 VOPC (Analysis)

Il VOPC è il diagramma delle classi di analisi che partecipano a UC-04, secondo il pattern **Boundary–Control–Entity**: le boundary `MarketplaceView` e `PaymentView` (con l'attore `Cliente`), il control unico `OrdinaProdottoController` e le entity `Ordine`, `Prodotto`, `BuonoPromozionale`, `Utente`. La notifica a venditore e compratore (pattern `Observer`) è rappresentata dal `OrdineEventPublisher` (subject singleton), dal DTO `OrdineEvent` e dagli observer `VenditoreNotifier` e `UtenteNotifier`. I Bean e i DAO appartengono al design-level e non compaiono qui.

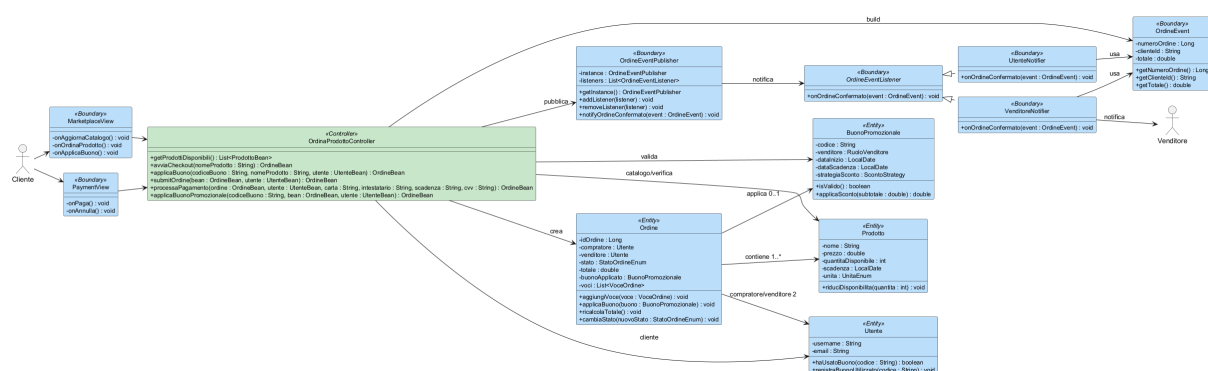


Figura 3.1: VOPC (analisi BCE) di UC-04: Boundary, Control ed Entity.

3.2 Class Diagram (Design-level)

Il design-level introduce interfacce, tipi Java, eccezioni e pattern GoF per gestire estensibilità e persistenza dinamica (NFR-01). Gli schemi dei pattern utilizzati sono mostrati di

seguito.

3.2.1 Controller

Per ogni caso d'uso esiste un *unico controller* che coordina la logica di business (DAO, buono, pagamento, submit) e le operazioni di orchestrazione esposte alle boundary con scambi su Bean. La boundary (MarketplaceView, PaymentView) gestisce l'interazione della UI tramite metodi privati `on...` e delega le operazioni di dominio al controller.

- **MarketplaceView / PaymentView**: gestione schermate (eventi UI in metodi `on...`) e messaggi user-friendly;
- **OrdinaProdottoController**: unico controller del caso d'uso, con la logica di business (buono 4a, pagamento 6/6a, submit) e le operazioni di checkout (`avviaCheckout`, `applicaBuono`).

Questa struttura rende le boundary intercambiabili (JavaFX o CLI) senza toccare la logica di dominio. Il flusso reale è: boundary (gestione schermate) → controller (business) → DAO.

Il diagramma delle classi di progettazione (design-level) mostra tutte le classi partecipanti di UC-04 con i pattern GoF applicati: la persistenza tramite **Abstract Factory** (**DAOFactory** e le concrete factory), la scontistica del buono tramite **Strategy**, e la notifica al venditore tramite **Observer**. Boundary blu/azzurro, controller verde, entity e interfacce nei toni di supporto.

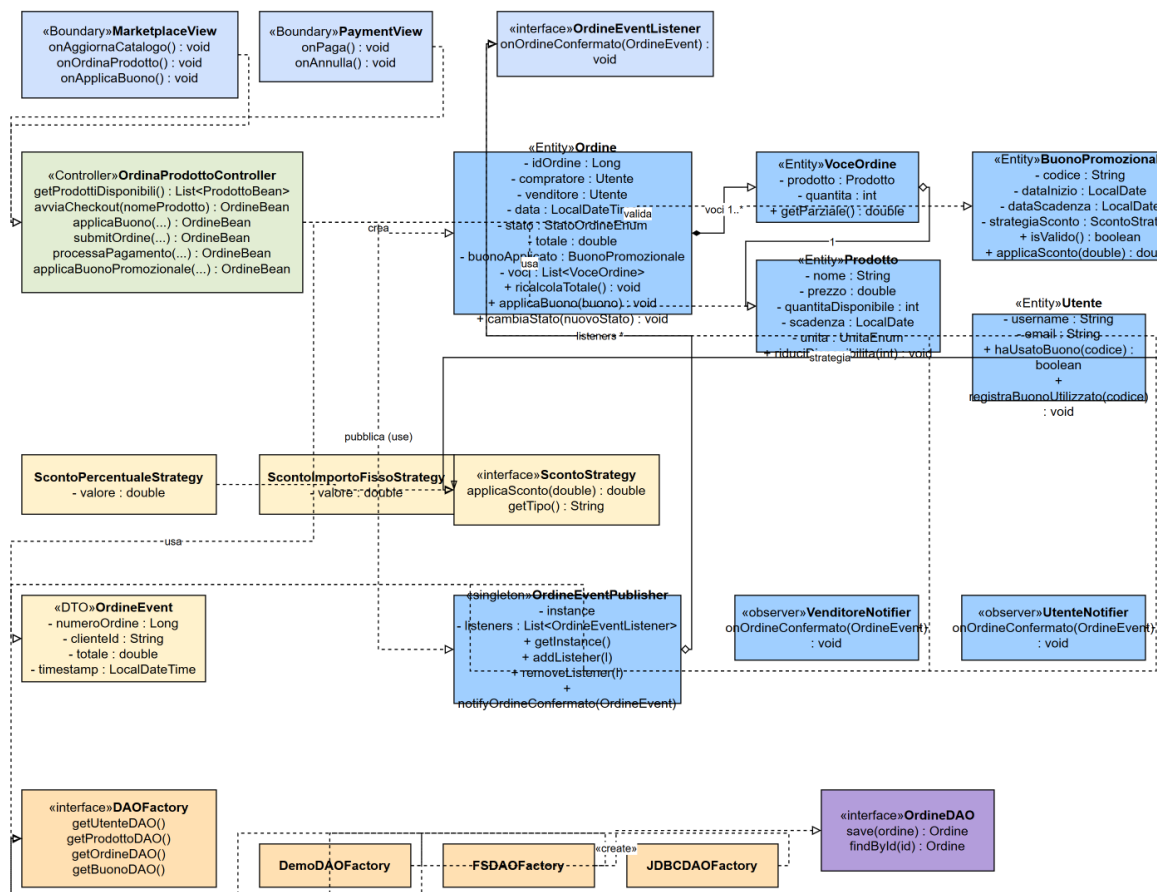


Figura 3.2: Class diagram (design-level) di UC-04: boundary, controller, entity e pattern GoF.

3.2.2 Validazione input e gestione errori

La validazione sintattica dell'input utente avviene nei setter dei Bean (Fail Fast, secondo il pattern BCE) e invalidate(). I controller grafici ricevono le **String** raw dal form, costruiscono il Bean (che valida da solo) e espongono messaggi user-friendly sulla UI.

Le eccezioni di dominio (**BusinessValidationException** e **AutenticazioneException**) separano il messaggio per l'utente (**getUserMessage()**) dal dettaglio tecnico per il log (**getTechnicalMessage()**) e, quando utile, definiscono un codice errore simbolico (**getErrorCode()**). In questo modo la UI mostra testi chiari e il log conserva il contesto, senza esporre dettagli interni all'utente finale.

3.2.3 Design Patterns (GoF)

Il design-level introduce le interfacce, i tipi Java, le eccezioni e i pattern GoF per gestire estensibilità e persistenza dinamica (NFR-01). Per ogni pattern sono mostrati in ordine lo schema e la breve descrizione dell'applicazione.

Abstract Factory Pattern – Persistence Layer

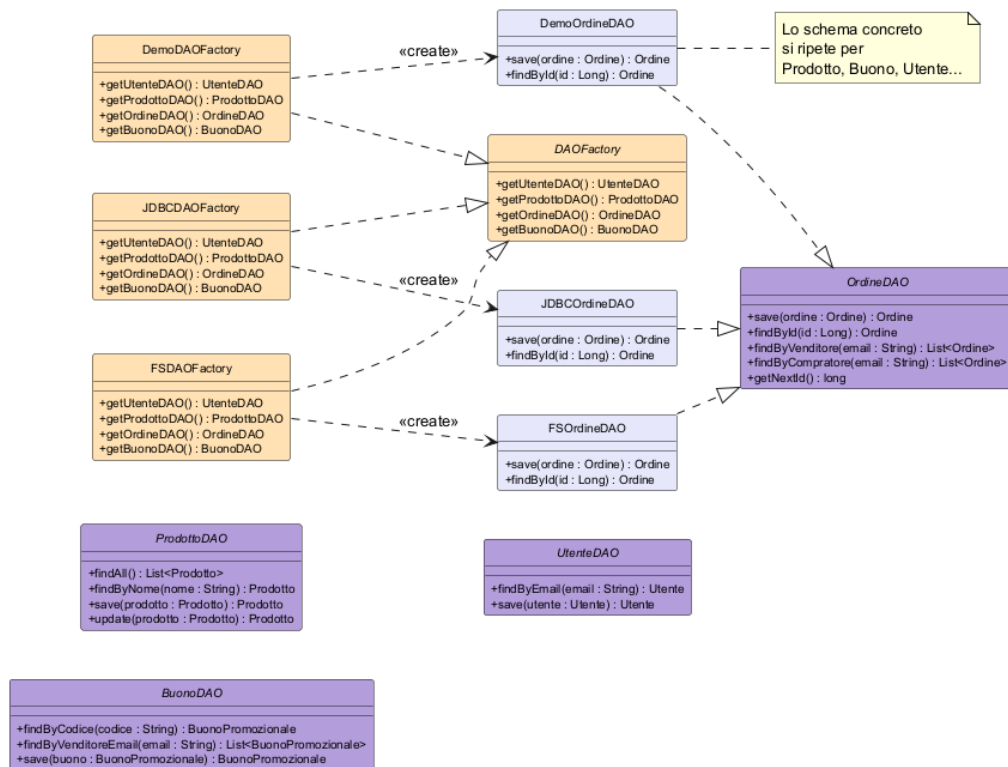


Figura 3.3: Abstract Factory della persistenza: interfacce DAO, concrete DAO e factory.

Il livello di persistenza è reso intercambiabile tramite il pattern **Abstract Factory**. L'`ADOFactory` definisce l'interfaccia per creare i vari Data Access Object (`demo/FS/JDBC`); in base alla configurazione `persistence_type`, le concrete factory (`DemoDAOFactory`, `FSDaoFactory`, `JDBCDAOFactory`) istanziano lo storage opportuno. Ciò consente di passare da database, file-system o memoria senza modificare il codice client (principio Open/Closed).

Strategy Pattern – Scontistica del buono

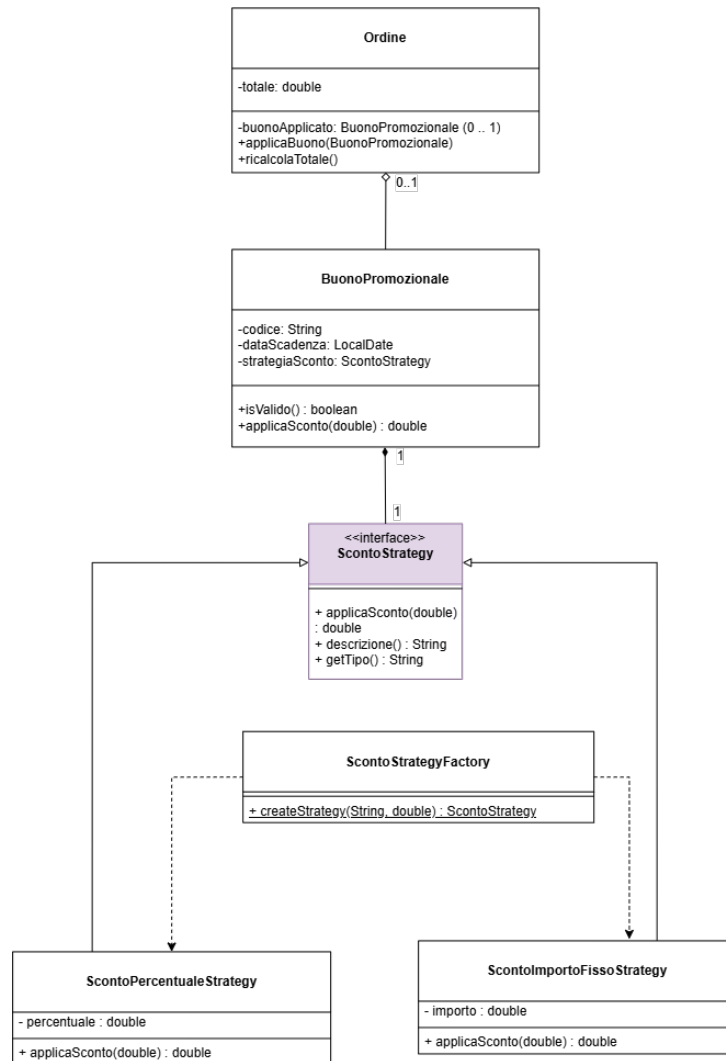


Figura 3.4: Strategy della scontistica del buono.

Gli sconti del buono sono calcolati tramite la **ScontoStrategy** (percentuale o importo fisso), usata da **Ordine** in `ricalcolaTotale()` e costruita da **ScontoStrategyFactory**. Aggiungere una nuova politica di sconto richiede solo una nuova classe senza modificare le classi esistenti.

Observer Pattern – Notification System

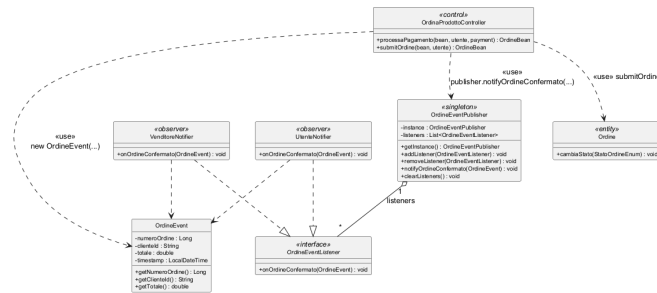


Figura 3.5: Observer della conferma dell'ordine: publisher singleton e DTO.

Il pattern Observer disaccoppia l'elaborazione dell'ordine dalla logica di notifica. La classe `OrdineEventPublisher` funge da *Subject Singleton*, gestendo una lista di `OrdineEventListener`. Alla conferma dell'ordine, il publisher invoca `onOrdineConfermato()` (compratore e venditore) passando il DTO `OrdineEvent` in sola lettura, mai l'entità di dominio: loose coupling e Open/Closed.

Singleton Pattern – Sessions and Mode

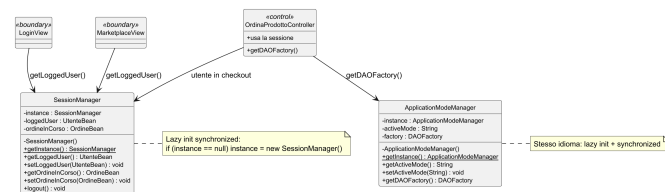


Figura 3.6: Singleton della sessione e della modalità.

`SessionManager` (sessione utente) e `ApplicationModeManager` (modalità applicativa) adottano un singleton classico con *lazy initialization* (`if (instance == null) instance = new X()`) e sincronizzazione su `getInstance()`, in linea coi progetti di riferimento: nessun costo percepibile in un client desktop, nessuna soppressione di warning per SonarCloud.

Facade Pattern – Checkout UC-04

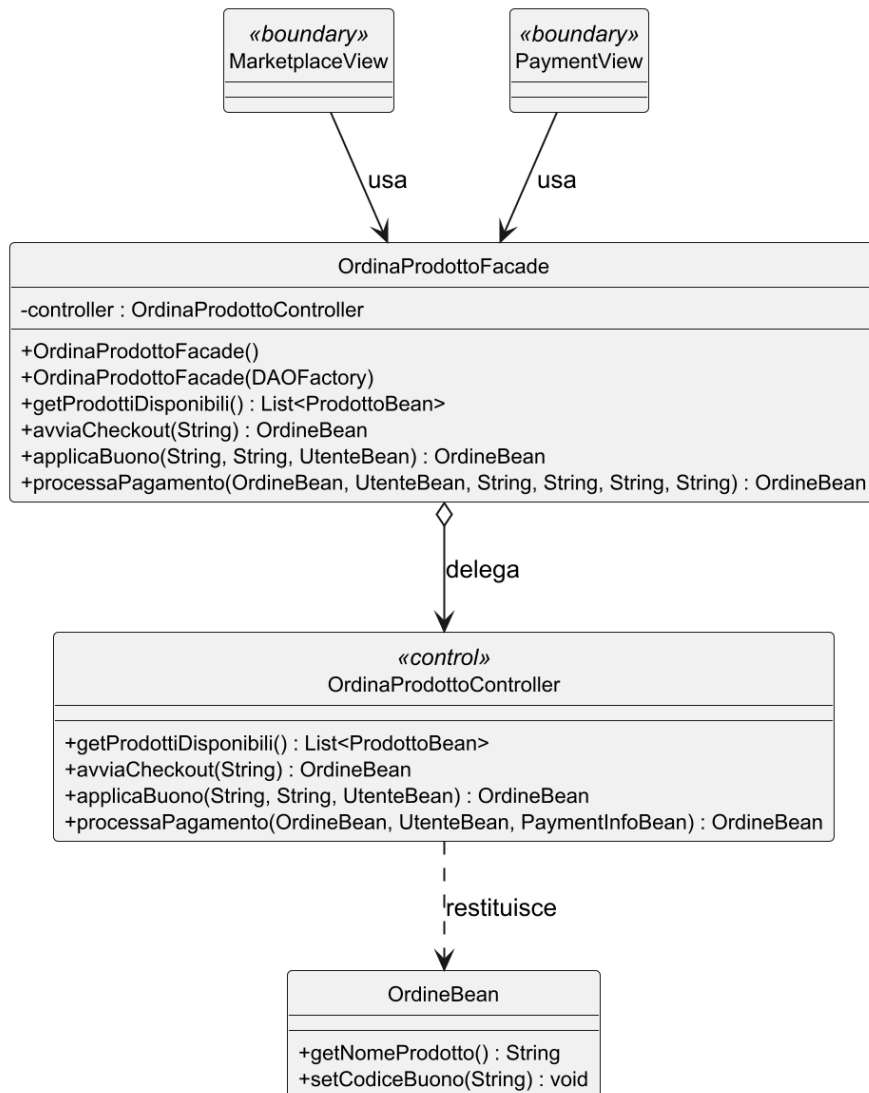


Figura 3.7: Facade del checkout di UC-04.

OrdinaProdottoFacade espone alla boundary un'unica interfaccia per il checkout (**getProdottiDisponibili**, **avviaCheckout**, **applicaBuono**, **processaPagamento**) e delega al controller le estensioni dell'use case (buono 4a, pagamento 6/6a, submit). La sottile boundary non conosce l'ordine di chiamata ai metodi del controller, che resta responsabile della singola regola di business.

3.3 Modellazione Comportamentale

3.3.1 Activity Diagram (UC-04)

Il flusso dell'UC-04: selezione prodotto, verifica disponibilità, applicazione opzionale del buono (4a), pagamento (6/6a) e registrazione dell'ordine.

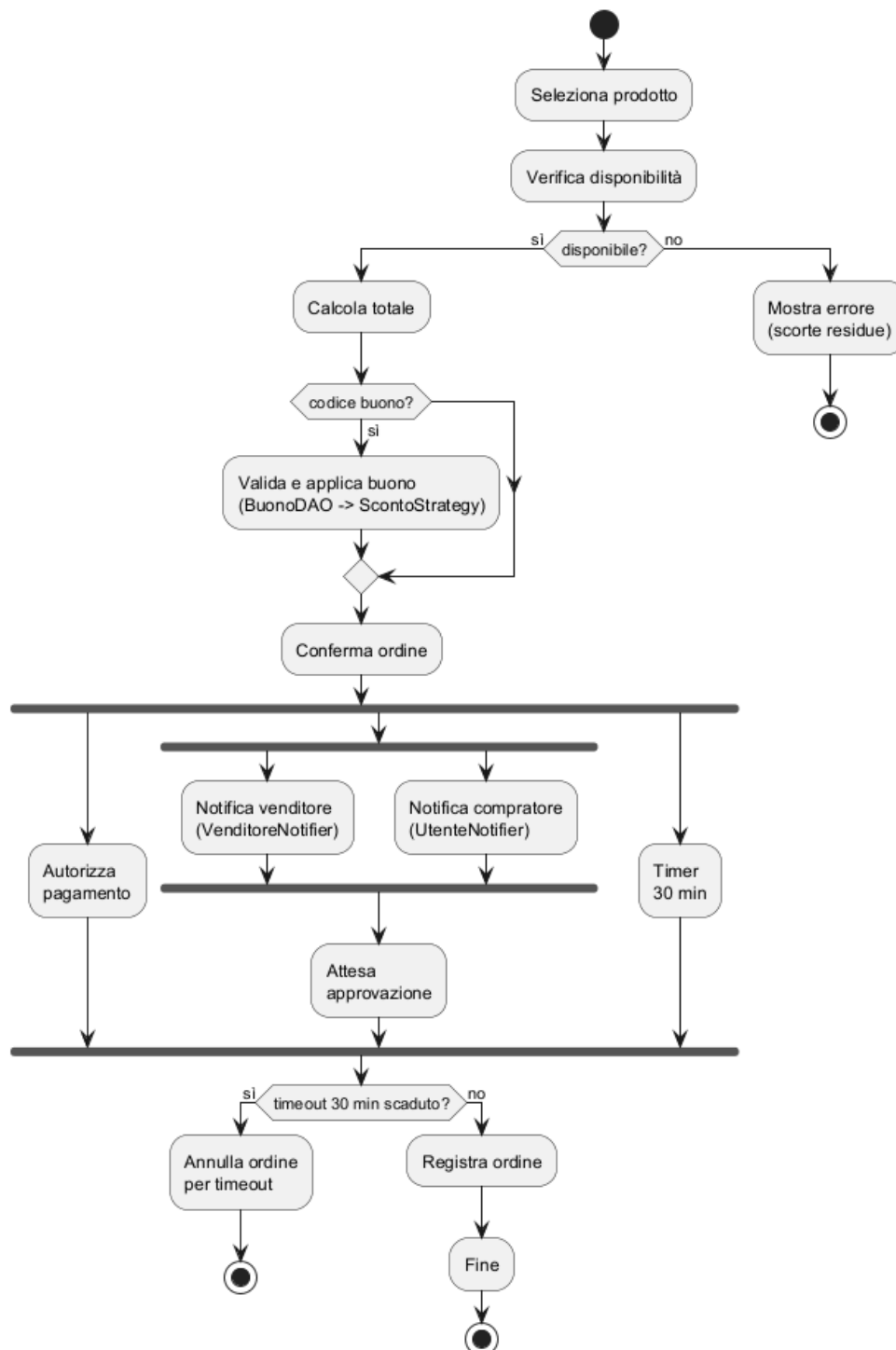


Figura 3.8: Activity diagram di UC-04: parallelismo di pagamento, notifica compratore/venditore (fork) e timer.

3.3.2 Sequence Diagram (UC-04)

Il sequence segue la *catena a gradino* (Lezione 32): `Cliente` autenticato crea `OrdineBean`, valida il buono (4a, frammento *opt*), e paga (`processaPagamento`) con autorizzazione

(PaymentGateway), riduzione scorte, salvataggio e notifica a venditore e compratore tramite Observer (OrdineEventPublisher + DTO OrdineEvent) (6a gestito inline).

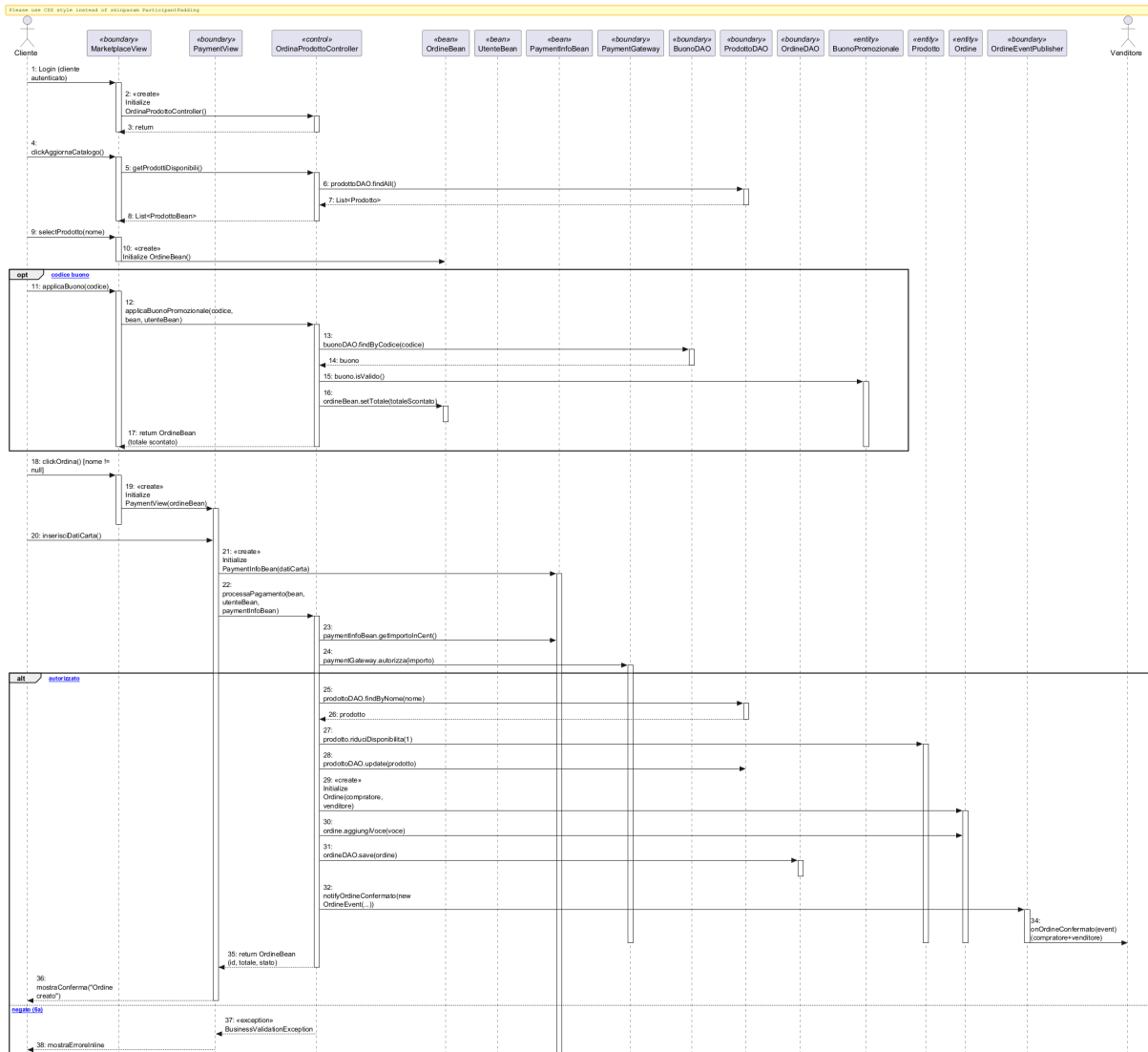


Figura 3.9: Sequence diagram di UC-04.

3.3.3 State Diagram (UC-04, navigazione UI)

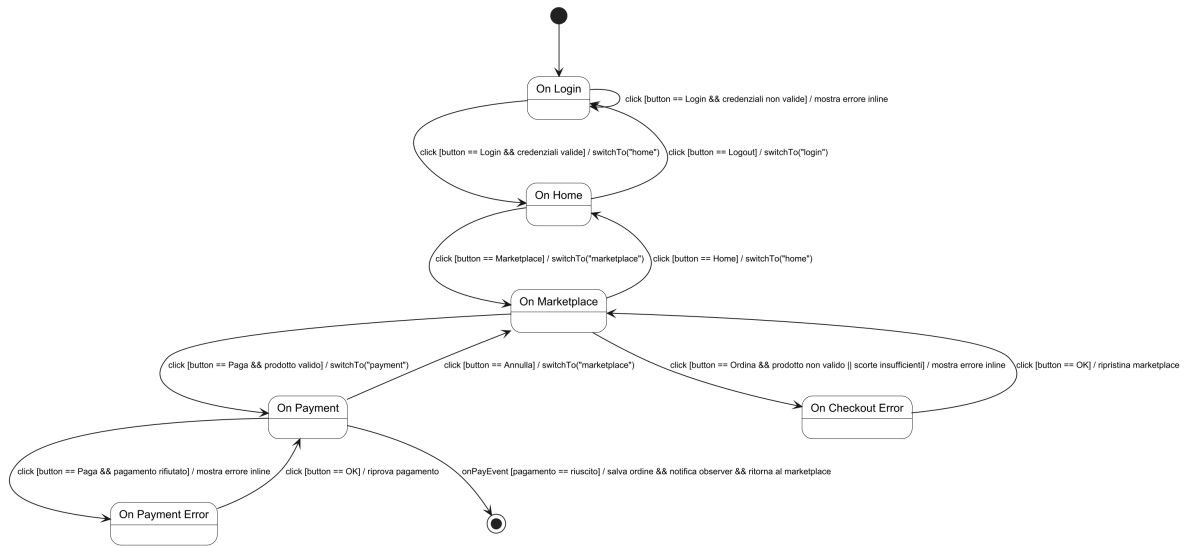


Figura 3.10: State diagram (navigazione UI) di UC-04.

3.3.4 State Diagram (ciclo di vita dell'ordine, BR-04)

Il lifecycle dell'entità `Ordine` è una macchina a stati validata da `Ordine.cambiaStato(nuovoStato)`: `CREATED` → `CONFIRMED` → `IN_DELIVERY` → `DELIVERED`, con `ANNULLED` ammesso solo da `CREATED` e `CONFIRMED`. Ogni transizione illegale è bloccata da `InvalidStateTransitionException`.

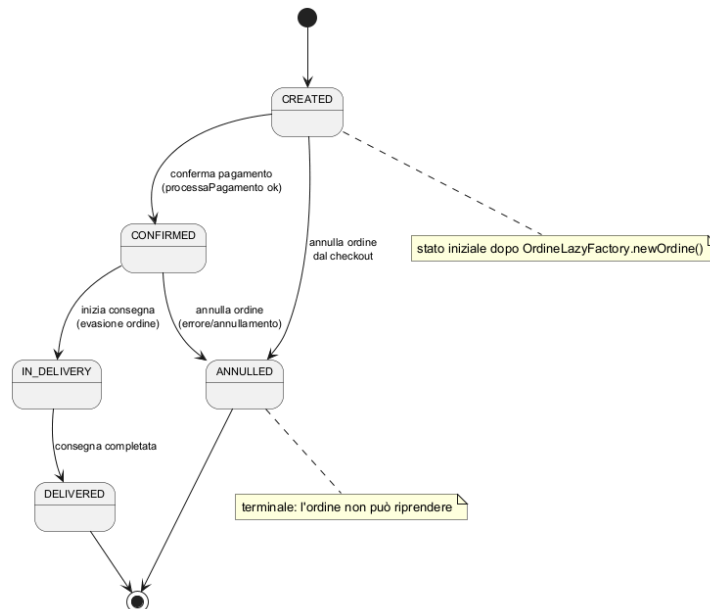


Figura 3.11: State diagram del lifecycle dell'ordine (BR-04).

Capitolo 4

Testing

La verifica del sistema è condotta con una suite di unit test **JUnit 5** eseguita da Maven e con copertura misurata da JaCoCo (quality gate su copertura di riga). Di seguito i comportamenti verificati, descritti per funzionalità.

Accesso all'area riservata. Abbiamo testato il controller dell'area riservata verificando che un utente autenticato recuperi correttamente i dati del proprio checkout e della propria sessione. I test garantiscono che le strutture restituite non siano mai `null` e che l'accesso ai dati riservati sia gestito correttamente per gli utenti autenticati.

Creazione e sottomissione dell'ordine. Abbiamo testato il flusso di ordinazione verificando che un prodotto venga selezionato e che l'ordine venga creato con il venditore corretto. I test garantiscono che un ordine con utente o prodotto nullo venga respinto e che la disponibilità di magazzino sia ridotta all'acquisto senza scorte negative.

Scontistica del buono (Strategy). Abbiamo testato la logica di scontistica verificando che un buono valido venga applicato al totale dell'ordine. I test garantiscono che codici vuoti, scaduti o non validi vengano respinti senza eccezioni non gestite e che il calcolo dello sconto non produca mai importi incoerenti.

Pagamento e autorizzazione. Abbiamo testato il sottosistema di pagamento verificando sia il ramo autorizzato sia quello rifiutato. I test garantiscono che un pagamento rifiutato sollevi l'eccezione applicativa corretta lasciando invariato l'ordine, e che dati di carta non validi (CVV, importo non positivo, ordine assente) vengano bloccati senza `NullPointerException`.

Autenticazione. Abbiamo testato il meccanismo di autenticazione verificando sia credenziali valide sia non valide. I test garantiscono che un login corretto popoli la sessione e

che email o password errate vengano rifiutate, con la `logout` che pulisce correttamente la sessione.

Ciclo di vita dell'ordine (macchina a stati BR-04). Abbiamo testato la macchina a stati dell'ordine verificando la sequenza `CREATED` $\rightarrow$ `CONFIRMED` $\rightarrow$ `IN_DELIVERY` $\rightarrow$ `DELIVERED`. I test garantiscono che una transizione illegale venga bloccata dall'eccezione di stato invalido.

Notifica Observer. Abbiamo testato il pattern Observer verificando che il publisher notifichi gli observer registrati quando un ordine viene confermato. I test garantiscono che l'evento (`OrdineEvent`) venga consegnato agli observer tramite il DTO in sola lettura, mai l'entità di dominio.

Persistenza (Demo e File System). Abbiamo testato i livelli di persistenza verificando il ciclo crea-leggi-aggiorna di utenti, prodotti e ordini. I test garantiscono che la persistenza *Demo* (in-memory) e *File System* mantengano i dati in modo coerente, incluso il recupero dei prodotti per nome.

Capitolo 5

Exceptions

Il sistema implementa una gerarchia di eccezioni custom basata su una classe base astratta, `CiboAmicoException`, che estende `java.lang.Exception` (checked), così da forzare la gestione esplicita degli errori di dominio.

Exception Chaining (Wrapping). Il costruttore (`String message, Throwable cause`) permette di:

- aggiungere contesto semantico all'errore;
- preservare lo stack trace originale;
- mantenere la catena causale per il debugging.

BusinessValidationException. Sotto-classe sollevata quando l'input viola una regola di business (scorte esaurite, buono non valido, pagamento rifiutato, campi carta non validi). Le boundary la catturano e mostrano il messaggio all'utente in una `Label`.

InvalidStateTransitionException. Specializzazione per le transizioni di stato non previste: la macchina a stati BR-04 (`Ordine.cambiaStato()`) blocca ogni operazione illegale.

AutenticazioneException. Specializzazione per gli errori di autenticazione (credenziali errate, email o password non valide), usata dal controller di autenticazione e dalla validazione dei bean.

DAOException. Sotto-classe per gli errori durante l'accesso alla persistenza: gli errori infrastrutturali (es. `SQLException`) sono incapsulati dai DAO e non si propagano alle view.

PaymentRejectedException. Eccezione del sottosistema di pagamento (`pattern.payment`), sollevata quando la transazione non viene autorizzata.

Messaggi centralizzati. I testi degli errori sono raccolti in due enum: `ExceptionMessagesEnum` (dettagli tecnici per il log) e `UserErrorMessagesEnum` (messaggi mostrati all'utente), evitando di duplicare stringhe nei punti di lancio.

Capitolo 6

Persistenza

La persistenza è intercambiabile a runtime (NFR-01): tre backend mutuamente esclusivi sono selezionati tramite il pattern **Abstract Factory** (`DAOFactory` + una concrete factory per modalità).

6.1 Layer di persistenza DB (MySQL/JDBC)

Il livello di persistenza DB utilizza **MySQL** come DBMS relazionale. La gestione dei dati è affidata a classi DAO specifiche (es. `JDBCUserDAO`, `JDBCOrdineDAO`), che eseguono query SQL per le operazioni CRUD sulle entità.

Entità principali:

- **Utenti:** registrati, con i ruoli (Cliente e Venditore);
- **Ordini:** record transazionali con stato, totale e voci;
- **Prodotti:** articoli del marketplace (catalogo e inventario);
- **Buoni:** codici sconto (percentuale o importo fisso).

Le transazioni sono atomiche (`ConnectionManager` usa `setAutoCommit(false)` con `commit/rollback` per la riduzione scorte).

Configurazione: il sistema è configurato via il file `config.properties` impostando `persistence_type = DB` e fornendo le credenziali MySQL necessarie.

6.2 Layer di persistenza FS (File System)

Il layer di persistenza FS memorizza i dati in file **JSON** nella cartella `data/`. Ogni entità è mappata su un file dedicato (es. `utenti.json`, `ordini.json`, `prodotti.json`, `inventario.json`).

Gestione: la classe `GsonConfig` fornisce un serializzatore JSON coerente (helper per `LocalDate/LocalDateTime`) usato da tutti i DAO FS per garantire la lettura/scrittura corretta dei record.

Configurazione: la modalità è attivata impostando `persistence_type = FS` nel file di configurazione.

6.3 Layer di persistenza DEMO (In-Memory)

Il layer di persistenza DEMO mantiene i dati esclusivamente nella memoria volatile (RAM), senza storage permanente. Le strutture sono popolate da valori predefiniti all'avvio e cancellate al termine dell'esecuzione.

Casi d'uso: è pensato per il testing unitario, lo sviluppo rapido e le dimostrazioni che non richiedono configurazione di database.

Configurazione: la modalità è attivata impostando `persistence_type = DEMO` (default).

Capitolo 7

Video e Link

7.1 Repository GitHub

<https://github.com/Damiano-o/ciboamico.git>

7.2 SonarCloud

<https://sonarcloud.io/organizations/ciboamico-ispw>

7.3 Video dimostrativo

Da inserire prima della consegna.